

## 1. FlatList vs ScrollView

When rendering lists of items, mobile devices have limited memory and processing power compared to web browsers. Choosing the right component makes a significant difference in app performance.

### ScrollView

ScrollView renders all of its child components at once as soon as the screen loads.

- **Best used for:** Short, fixed content (like forms, settings pages, or small lists with < 20 items).
- **The Problem:** If you render 1,000 items in a ScrollView, React Native creates 1,000 native views at startup. This leads to slow initial renders, high memory usage, and UI lag.

JavaScript

```
import { ScrollView, Text, View } from 'react-native';
```

```
// ✗ BAD for large lists - renders all items immediately
const BadList = () => (
  <ScrollView>
    {Array.from({ length: 1000 }, (_, i) => (
      <View key={i} style={{ padding: 20, borderBottomWidth: 1 }}>
        <Text>Item #{i + 1}</Text>
      </View>
    ))}
  </ScrollView>
);
```

```
export default BadList;
```

### FlatList

FlatList uses **virtualization**. It only renders the items currently visible on screen (plus a small buffer) and unmounts or recycles items that go off-screen.

- **Best used for:** Large, dynamic, or infinite lists (like social media feeds, chat messages, or product catalogs).
- **Key Features:** Built-in virtualization, pagination (onEndReached), pull-to-refresh, header/footer components, and empty list states out of the box.

JavaScript

```
import { FlatList, Text, View } from 'react-native';

// ✔ GOOD - only renders visible items on screen
const GoodList = () => {
  const data = Array.from({ length: 1000 }, (_, i) => ({
    id: i.toString(),
    title: `Item #${i + 1}`,
  }));

  return (
    <FlatList
      data={data}
      renderItem={({ item }) => (
        <View style={{ padding: 20, borderBottomWidth: 1 }}>
          <Text>{item.title}</Text>
        </View>
      )}
      keyExtractor={item => item.id}
    />
  );
};

export default GoodList;
```

## 2. Core Props: renderItem & keyExtractor

FlatList relies on two essential props to display and manage your list data properly.

### renderItem

renderItem is a function that receives an object containing { item, index, separators } and returns a JSX element to display that specific row.

JavaScript

```
renderItem={({ item, index }) => (
  <View>
    <Text>{index + 1}. {item.name}</Text>
  </View>
)}
```

## keyExtractor

keyExtractor takes an item from your data array and returns a **unique string** key for it.

- **Why it matters:** React relies on keys to track which items have changed, been added, or removed. Without unique keys, React re-renders the whole list whenever a change occurs, causing flickering and performance drops.
- **Important:** Keys **must be strings**.

JavaScript

```
// Converting numeric IDs to strings safely  
keyExtractor={item => item.id.toString()}
```

## 3. Lazy Loading & Infinite Scroll

For applications fetching data from an API, loading thousands of items at once wastes bandwidth and battery life. **Lazy loading** fetches data in smaller chunks (pages) as the user scrolls down.

### Essential Props for Infinite Scroll

1. **onEndReached:** A callback function triggered when the scroll position gets close to the end of the list.
2. **onEndReachedThreshold:** A number between 0 and 1 that defines how close to the bottom (in screen lengths) the scroll position must be before **onEndReached** fires. For example, 0.5 means trigger when the user is half a screen length away from the bottom.

### Basic Lazy Loading Implementation

JavaScript

```
import React, { useState, useEffect } from 'react';  
import { FlatList, Text, View, ActivityIndicator, StyleSheet } from 'react-native';
```

```
const LazyList = () => {  
  const [items, setItems] = useState([]);  
  const [page, setPage] = useState(1);  
  const [loading, setLoading] = useState(false);  
  const [hasMore, setHasMore] = useState(true);
```

```

const fetchItems = async (pageNum) => {
  // Prevent duplicate calls if already loading or no items left
  if (loading || (!hasMore && pageNum !== 1)) return;

  setLoading(true);
  try {
    const response = await fetch(
      `https://jsonplaceholder.typicode.com/posts?_page=${pageNum}&_limit=15`
    );
    const newItems = await response.json();

    if (newItems.length === 0) {
      setHasMore(false);
    } else {
      setItems(prev => [...prev, ...newItems]); // Append new page items
      setPage(pageNum + 1);
    }
  } catch (error) {
    console.error(error);
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  fetchItems(1);
}, []);

return (
  <FlatList
    data={items}
    keyExtractor={item => `item-${item.id}`}
    renderItem={({ item }) => (
      <View style={styles.card}>
        <Text style={styles.title}>{item.title}</Text>
      </View>
    )}
    onEndReached={() => fetchItems(page)}
    onEndReachedThreshold={0.5}
    ListFooterComponent={

```

```

      loading ? <ActivityIndicator size="large" color="#007AFF" /> : null
    }
  />
);
};

const styles = StyleSheet.create({
  card: { padding: 16, borderBottomWidth: 1, borderBottomColor: '#eee' },
  title: { fontSize: 16, fontWeight: '500' },
});

export default LazyList;

```

#### 4. Header, Footer, & Empty Components

Inserting elements like search bars or bottom loading spinners directly outside a FlatList inside a scrollable layout causes nesting errors. Instead, FlatList offers dedicated props to inject UI components directly inside its internal scroll hierarchy.

- **ListHeaderComponent:** Renders at the very top of the list (e.g., Search bar, summary banners).
- **ListFooterComponent:** Renders at the bottom of the list (e.g., Loading spinners, "End of items" notice).
- **ListEmptyComponent:** Renders when the data array is empty (e.g., Empty state messages, retry buttons).

#### 5. Production-Ready Example

Here is a full example bringing together filtering, pagination, pull-to-refresh, headers, footers, and empty states.

```

JavaScript
import React, { useState, useEffect } from 'react';
import {
  FlatList,
  Text,
  View,
  TextInput,
  ActivityIndicator,
  RefreshControl,
  TouchableOpacity,

```

```
SafeAreaView,  
StyleSheet,  
} from 'react-native';
```

```
const CompleteListExample = () => {  
  const [items, setItems] = useState([]);  
  const [page, setPage] = useState(1);  
  const [loading, setLoading] = useState(false);  
  const [refreshing, setRefreshing] = useState(false);  
  const [hasMore, setHasMore] = useState(true);  
  const [searchTerm, setSearchTerm] = useState("");  
  
  // 1. Fetch data with pagination  
  const fetchItems = async (pageNum, reset = false) => {  
    if (loading || (!hasMore && !reset)) return;  
  
    setLoading(true);  
    try {  
      const response = await fetch(  
        `https://jsonplaceholder.typicode.com/posts?_page=${pageNum}&_limit=10`  
      );  
      const newItems = await response.json();  
  
      if (reset) {  
        setItems(newItems);  
      } else {  
        setItems(prev => [...prev, ...newItems]);  
      }  
  
      setHasMore(newItems.length > 0);  
      setPage(reset ? 2 : pageNum + 1);  
    } catch (error) {  
      console.error('Fetch error:', error);  
    } finally {  
      setLoading(false);  
      setRefreshing(false);  
    }  
  };  
  
  // 2. Initial load
```

```
useEffect(() => {  
  fetchItems(1);  
}, []);
```

```
// 3. Pull-to-Refresh handler  
const handleRefresh = () => {  
  setRefreshing(true);  
  setHasMore(true);  
  fetchItems(1, true);  
};
```

```
// 4. Client-side search filtering  
const filteredItems = items.filter(item =>  
  item.title.toLowerCase().includes(searchTerm.toLowerCase())  
);
```

```
// Renderers for Header, Footer, Empty state, and Row  
const renderHeader = () => (  
  <View style={styles.headerContainer}>  
    <Text style={styles.headerTitle}>Posts Feed</Text>  
    <TextInput  
      style={styles.searchInput}  
      placeholder="Search posts..."  
      value={searchTerm}  
      onChangeText={setSearchTerm}  
    />  
  </View>  
);
```

```
const renderFooter = () => {  
  if (!loading) return null;  
  return (  
    <View style={styles.footerContainer}>  
      <ActivityIndicator size="small" color="#007AFF" />  
      <Text style={styles.footerText}>Loading more items...</Text>  
    </View>  
  );  
};
```

```
const renderEmpty = () => (  
  <View style={styles.emptyContainer}>  
    <Text style={styles.emptyText}>No posts found.</Text>  
  </View>  
);
```

```

<View style={styles.emptyContainer}>
  <Text style={styles.emptyText}>No posts found.</Text>
  <TouchableOpacity style={styles.retryButton} onPress={handleRefresh}>
    <Text style={styles.retryText}>Reload List</Text>
  </TouchableOpacity>
</View>
);

```

```

const renderItem = ({ item }) => (
  <View style={styles.card}>
    <Text style={styles.cardTitle}>{item.title}</Text>
    <Text style={styles.cardBody} numberOfLines={2}>
      {item.body}
    </Text>
  </View>
);

```

```

return (
  <SafeAreaView style={styles.container}>
    <FlatList
      data={filteredItems}
      renderItem={renderItem}
      keyExtractor={item => `post-${item.id}`}
      ListHeaderComponent={renderHeader}
      ListFooterComponent={renderFooter}
      ListEmptyComponent={renderEmpty}
      refreshControl={
        <RefreshControl refreshing={refreshing} onRefresh={handleRefresh} />
      }
      onEndReached={() => {
        if (!loading && hasMore) fetchItems(page);
      }}
      onEndReachedThreshold={0.3}
      // Performance Props
      initialNumToRender={10}
      maxToRenderPerBatch={10}
      windowSize={10}
    />
  </SafeAreaView>
);

```



```

});

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f8f9fa' },
  headerContainer: { padding: 16, backgroundColor: '#fff', borderBottomWidth: 1,
borderColor: '#eee' },
  headerTitle: { fontSize: 22, fontWeight: 'bold', marginBottom: 8 },
  searchInput: { height: 40, backgroundColor: '#f0f0f0', borderRadius: 8,
paddingHorizontal: 12 },
  card: { backgroundColor: '#fff', padding: 16, marginHorizontal: 16, marginTop:
12, borderRadius: 8 },
  cardTitle: { fontSize: 16, fontWeight: 'bold', textTransform: 'capitalize' },
  cardBody: { fontSize: 14, color: '#666', marginTop: 4 },
  footerContainer: { padding: 20, alignItems: 'center' },
  footerText: { marginTop: 6, color: '#888', fontSize: 12 },
  emptyContainer: { padding: 40, alignItems: 'center' },
  emptyText: { fontSize: 16, color: '#888', marginBottom: 12 },
  retryButton: { backgroundColor: '#007AFF', paddingHorizontal: 16,
paddingVertical: 8, borderRadius: 6 },
  retryText: { color: '#fff', fontWeight: '600' },
});

export default CompleteListExample;

```

## Performance Optimization Checklist

To ensure your lists run smoothly at 60 FPS:

1. **initialNumToRender:** Set this to the exact number of items required to fill the initial screen area.
2. **maxToRenderPerBatch:** Controls how many items are rendered per incremental batch during scrolling. Lower numbers reduce render lag, while higher numbers prevent blank spaces from appearing when scrolling rapidly.
3. **windowSize:** Controls how many screens' worth of content stays mounted in memory outside the viewport (default is 21). Lowering this number (e.g., to 10) saves memory on low-end devices.
4. **Use Lightweight Component Trees:** Keep your row items simple. Avoid complex calculations or inline functions inside `renderItem`.